

Marco 2

Driver Linux em Assembly ARM

Comunicação HPS FPGA via Lightweight Bridge

DE1-SoC · Cyclone V · ARM Cortex-A9 · UEFS TEC499 · 2026.1

Componentes : Jones Bacelar e Rogério
Cerqueira

Objetivo

HPS e FPGA

- O objetivo principal é **desenvolver, integrar e validar uma infraestrutura de software de baixo nível (Driver Linux)** que permite ao processador ARM (HPS) controlar e enviar dados de forma estável para o co-processador acelerador de rede neural ELM, desenvolvido no Marco 1 dentro da FPGA da placa DE1-SoC.

HPS - FPGA

HPS e FPGA dentro do mesmo encapsulamento

HPS — Hard Processor System

- ARM Cortex-A9 dual-core 925 MHz
- RAM DDR3 1 GB
- Linux rodando aqui
- USB, Ethernet, SD card
- Executa elm_exec.S

Lightweight
Bridge

0xFF200000

FPGA — Lógica Reconfigurável

- CoProcessor ELM (rede neural)
- 3 PIOs: data_in, data_out, ctrl
- BRAM com pesos W_in, bias, beta
- Configurada pelo soc_system.sof

Os 3 PIOs — Ponte HPS FPGA

Configurados no Platform Designer (Qsys).

data_in

32 bits · HPS → FPGA

0xFF200040

HPS envia instrução ao CoProcessor

data_out

32 bits · FPGA → HPS

0xFF200050

CoProcessor retorna resultado ao HPS

ctrl

3 bits · HPS → FPGA

0xFF200060

bit0=Enable bit1=ClrOp bit2=Reset

Bridge base: 0xFF200000 (fixo no Cyclone V) + offset do PIO = endereço físico

Endereços Físicos

Bridge base fixo no Cyclone V + offset definido no Platform Designer

0xFF200000 (bridge base) + offset do PIO = endereço físico

data_in

0xFF200000

+ 0x40

0xFF200040

HPS → FPGA

data_out

0xFF200000

+ 0x50

0xFF200050

FPGA → HPS

ctrl

0xFF200000

+ 0x60

0xFF200060

HPS → FPGA

```
ldr r4, [r0, #ELM_OFF_DATA_IN] @ r4 = 0xFF200040 → escreve instrução
```

Conjunto de Instruções (ISA)

6 instruções implementadas em Assembly — codificadas nos 32 bits de data_in

Instrução	OPCode	Função Assembly	Descrição
Store Image	000	enc_store_img()	Armazena pixel da imagem na BRAM
Store Weights Addr	001	enc_store_w_addr()	Define endereço do próximo peso
Store Weights Value	010	enc_store_w_value()	Armazena valor do peso (par com Addr)
Store Bias	011	enc_store_bias()	Armazena bias na BRAM
Store Beta	100	enc_store_beta()	Armazena beta na BRAM
Start	101	enc_start()	Inicia a inferência

Layout dos 32 bits — Instrução data_in

Cada instrução é uma palavra de 32 bits enviada ao PIO data_in (0xFF200040)

STORE_IMG (000)



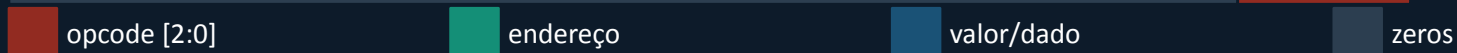
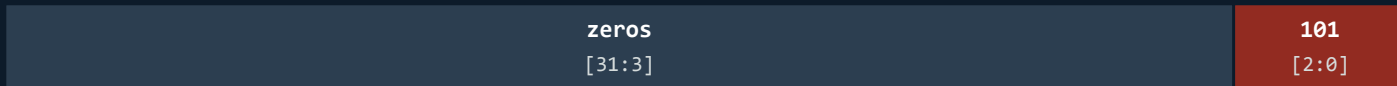
STORE_BIAS (011)



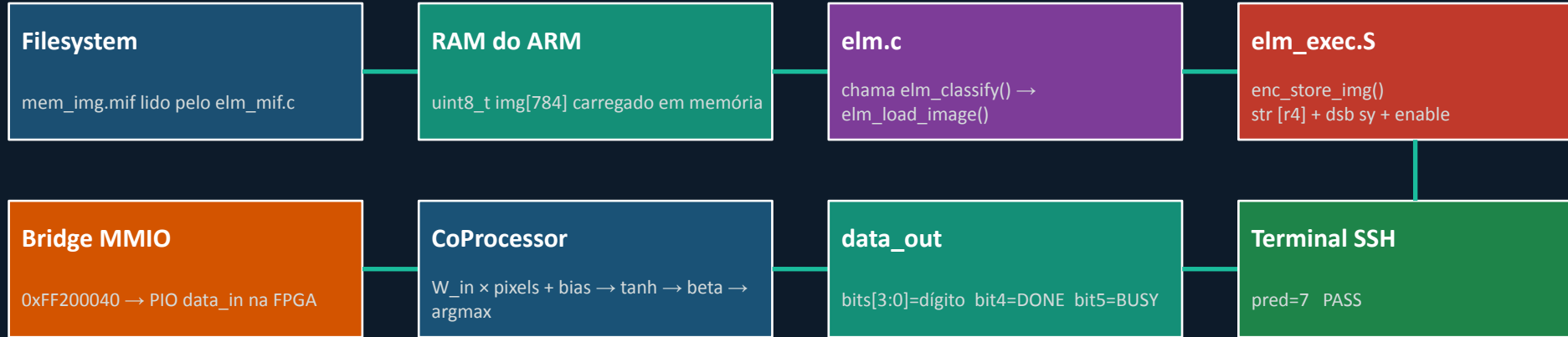
STORE_BETA (100)



START (101)



Fluxo Completo de uma Inferência



Estrutura dos Arquivos do Driver

Cada arquivo tem uma responsabilidade clara e isolada

elm_exec.S

Assembly ARM

9 funções. Handshake, encoders da ISA, reset, dsb sy.

elm_mif.c

Parser

Lê arquivos .mif do filesystem e converte para arrays na RAM do ARM.

hps_0.h

Endereços

DATA_IN_BASE=0x40, DATA_OUT_BASE=0x50, CTRL_BASE=0x60.
Criado manualmente (Quartus 24.1).

elm.c

Driver C

Aguarda o elm_open(), elm_classify(). Abre /dev/mem, faz mmap da bridge.

elm_proto.h

Protocolo

Opcodes (000-101), shifts, máscaras, dimensões da rede.
Compartilhado entre C e Assembly.

marco2_test.c

Teste

Carrega pesos, roda N inferências, mede latência, imprime PASS/FAIL.

elm_exec.S

9 funções em Assembly ARM

Sequência elm_exec()

- 1. Espera BUSY cair (data_out bit5)
- 2. Limpa erro residual se houver
- 3. Escreve instrução em data_in
- 4. dsb sy — barreira de memória
- 5. Sobe enable em ctrl
- 6. Polling DONE / ERROR
- 7. Abaixa enable
- 8. Espera BUSY cair (handshake OK)

Por que Assembly?

- Compilador C pode reordenar stores em MMIO
- Polling sem otimização indevida do -O2
- Acesso direto aos endereços físicos dos PIOs

Retorna: 0 = sucesso -1 = ERROR no hardware

Conclusão

- elm_exec.S implementa em Assembly ARM com acesso ao hardware
- 9 funções: handshake, reset, clear_error e 6 encoders da ISA
- Inferência correta e estável validada na placa em múltiplos dígitos

Co-processador: github.com/DestinyWolf/Problema_SD_2026_1